

Complete Solutions to NLP Practice Problems

Q1. CKY Table for "A student likes writing tests"

The CKY (Cocke-Kasami-Younger) algorithm is a bottom-up parsing algorithm that determines if a string can be generated by a given Context-Free Grammar. The grammar must first be in Chomsky Normal Form (CNF).

Grammar in Chomsky Normal Form (CNF)

- **Syntactic Rules:**

```
S    -> NP VP
VP   -> VBG NNS
VP   -> VBZ VP
VP   -> VBZ NP
NP   -> DT NN
NP   -> JJ NNS
```

- **Lexical Rules (Terminals):**

```
DT   -> 'A'
NN   -> 'student'
VBZ  -> 'likes'
VBG  -> 'writing'
JJ   -> 'writing'
NNS  -> 'tests'
```

CKY Parsing Table

The table is filled bottom-up. Each cell (i, j) contains the set of non-terminals that can generate the substring from word i to word j .

Sentence: A (w_1) student (w_2) likes (w_3) writing (w_4) tests (w_5)

j	1	2	3	4	5
i=1	{DT}	{NP}	$\emptyset$	$\emptyset$	{S}
i=2		{NN}	$\emptyset$	$\emptyset$	$\emptyset$
i=3			{VBZ}	$\emptyset$	{VP}
i=4				{VBG, JJ}	{NP, VP}
i=5					{NNS}

Step-by-Step Derivations

Span 1 (Diagonal):

- ‘table(1,1)’ for ”A”: $\{DT\}$ from rule ‘DT - $\bar{i}$ ’A’
- ‘table(2,2)’ for ”student”: $\{NN\}$ from rule ‘NN - $\bar{i}$ ’student’
- ‘table(3,3)’ for ”likes”: $\{VBZ\}$ from rule ‘VBZ - $\bar{i}$ ’likes’
- ‘table(4,4)’ for ”writing”: $\{VBG, JJ\}$ from rules ‘VBG - $\bar{i}$ ’writing’ and ‘JJ - $\bar{i}$ ’writing’
- ‘table(5,5)’ for ”tests”: $\{NNS\}$ from rule ‘NNS - $\bar{i}$ ’tests’

Span 2:

- ‘table(1,2)’ for ”A student”: Combine ‘table(1,1)’ and ‘table(2,2)’. Rule ‘NP - $\bar{i}$ DT NN’ applies, so we add $\{NP\}$.
- ‘table(4,5)’ for ”writing tests”: Combine ‘table(4,4)’ and ‘table(5,5)’. Rule ‘NP - $\bar{i}$ JJ NNS’ applies (‘JJ’ from ‘table(4,4)’), so we add $\{NP\}$. Rule ‘VP - $\bar{i}$ VBG NNS’ applies (‘VBG’ from ‘table(4,4)’), so we add $\{VP\}$.

Span 3:

- ‘table(3,5)’ for ”likes writing tests”: Combine ‘table(3,3)’-‘table(4,5)’ and ‘table(3,4)’-‘table(5,5)’. The only valid combination is ‘table(3,3)’ and ‘table(4,5)’. Rule ‘VP - $\bar{i}$ VBZ NP’ applies (‘VBZ’ from ‘table(3,3)’ and ‘NP’ from ‘table(4,5)’), so we add $\{VP\}$.

Span 5:

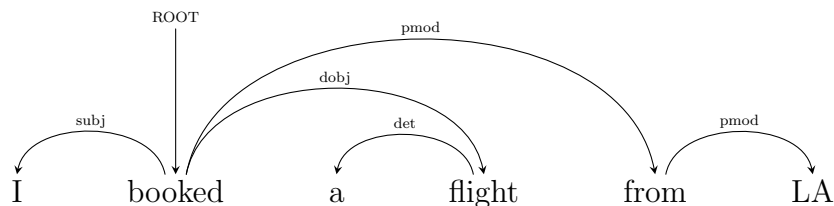
- ‘table(1,5)’ for the full sentence: We check all split points. The successful split is ‘k=2’: ‘table(1,2)’ and ‘table(3,5)’. Rule ‘S - $\bar{i}$ NP VP’ applies (‘NP’ from ‘table(1,2)’ and ‘VP’ from ‘table(3,5)’), so we add $\{S\}$.

Since the start symbol **S** is in the final cell ‘table(1,5)’, the sentence is recognized as grammatical.

Q2. Dependency Parsing Transitions

The task is to determine the sequence of transitions (using the Arc-Standard system) that produces the given dependency graph for the sentence "I booked a flight from LA".

Target Dependency Graph



Transition Trace Table

The Arc-Standard system uses three actions: SHIFT, LEFT-ARC, and RIGHT-ARC.

Step	Transition	Stack	Buffer
0	Initial	‘[ROOT]‘	‘[I, booked, a, flight, from, LA]‘
1	‘SHIFT‘	‘[ROOT, I]‘	‘[booked, a, flight, from, LA]‘
2	‘SHIFT‘	‘[ROOT, I, booked]‘	‘[a, flight, from, LA]‘
3	‘LEFT-ARC(subj)‘	‘[ROOT, booked]‘	‘[a, flight, from, LA]‘
4	‘SHIFT‘	‘[ROOT, booked, a]‘	‘[flight, from, LA]‘
5	‘SHIFT‘	‘[ROOT, booked, a, flight]‘	‘[from, LA]‘
6	‘LEFT-ARC(det)‘	‘[ROOT, booked, flight]‘	‘[from, LA]‘
7	‘RIGHT-ARC(dobj)‘	‘[ROOT, booked]‘	‘[from, LA]‘
8	‘SHIFT‘	‘[ROOT, booked, from]‘	‘[LA]‘
9	‘SHIFT‘	‘[ROOT, booked, from, LA]‘	‘[]‘
10	‘RIGHT-ARC(pmod)‘	‘[ROOT, booked, from]‘	‘[]‘
11	‘RIGHT-ARC(pmod)‘	‘[ROOT, booked]‘	‘[]‘
12	‘RIGHT-ARC(root)‘	‘[ROOT]‘	‘[]‘
13	Done	‘[ROOT]‘	‘[]‘

The final set of arcs created is: (booked $\leftarrow$ I), (flight $\leftarrow$ a), (booked $\rightarrow$ flight), (from $\rightarrow$ LA), (booked $\rightarrow$ from), (ROOT $\rightarrow$ booked), which exactly matches the target graph.

Q3. Symmetric Delete Spelling Correction

Core Principle

The Symmetric Delete algorithm is an efficient way to find candidate corrections for a misspelled word. It works on the principle that if two words are a single edit distance apart (due to insertion, deletion, or substitution), they will share a common string that is one deletion away from at least one of them. We generate all single-deletion variants for the typo and look for them in a pre-computed dictionary of single-deletion variants from correctly spelled words.

Application: ‘hoper’ vs. ‘hotel’

The typo ‘hoper’ for ‘hotel’ involves two substitutions (‘p’ for ‘t’, ‘r’ for ‘l’), so it has an edit distance of 2. The standard symmetric-delete method focuses on edit distance 1. Let’s first analyze a distance-1 case, like ‘hoter’ (typo) vs ‘hotel’ (correct).

Case 1: ‘hoter’ vs. ‘hotel’ (Edit Distance = 1)

- Single-deletions from typo ‘hoter’: {oter, hter, hoer, hotr, hote}
- Single-deletions from correct word ‘hotel’: {otel, htel, hoel, hotl, hote}

Result: A match is found. The string **hote** is common to both sets. Therefore, ‘hotel’ is correctly identified as a valid candidate for the typo ‘hoter’.

Why it doesn’t match ‘horse’ or ‘chore’

Case 2: ‘hoper’ vs. ‘horse’

- Single-deletions from typo ‘hoper’: {oper, hper, hoer, hopr, hope}
- Single-deletions from correct word ‘horse’: {orse, hrse, hose, hore, hors}

Result: There is **no common string** between the two sets. The algorithm correctly rules out ‘horse’ as a candidate.

Case 3: ‘hoper’ vs. ‘chore’

- Single-deletions from typo ‘hoper’: {oper, hper, hoer, hopr, hope}

- Single-deletions from correct word ‘**chore**’: {hore, core, chre, choe, chor}

Result: There is **no common string**, so ‘chore’ is also correctly ruled out as a candidate.

Q4. Bigram Language Model from a Paragraph

Part 1: Creating the Bigram Model

First, we tokenize the paragraph and compute unigram and bigram counts. The vocabulary is restricted to only the words in the paragraph. We will only list the counts relevant for the second part of the question.

Unigram Counts:

- $C(\text{she}) = 3$
- $C(\text{learned}) = 2$

Bigram Counts:

- $C(\text{she, learned}) = 1$
- $C(\text{learned, that}) = 1$
- $C(\text{learned, hert}) = 0$

From these counts, we can calculate the Maximum Likelihood Estimation (MLE) probabilities. For example:

$$P(\text{learned}|\text{she}) = \frac{C(\text{she, learned})}{C(\text{she})} = \frac{1}{3} \approx 0.333$$

Part 2: Detecting Errors in "She learned hert"

We analyze the phrase using our model.

1. Non-Word Error Detection: A non-word error is a word that is not in our vocabulary. We scan the paragraph and find that the word "****hert****" does not appear anywhere.

- **Conclusion:** Since the vocabulary is limited to the words in the paragraph, 'hert' is immediately flagged as a **non-word error**.

2. Real-Word / Contextual Error Detection: Even if 'hert' was a real word in our vocabulary, the bigram model would detect a contextual error. The probability of the bigram '(learned, hert)' is:

$$P(\text{hert}|\text{learned}) = \frac{C(\text{learned, hert})}{C(\text{learned})} = \frac{0}{2} = 0$$

- **Conclusion:** A zero-probability bigram is a strong signal of a contextual error. The model has learned from the corpus that the sequence "learned hert" is impossible, while a sequence like "learned that" is plausible. This allows the model to detect that the phrase is incorrect.

Q5. Word Vector Analogy Calculation

Concept

Word vectors learned by models like word2vec capture semantic relationships as linear offsets in a vector space. The classic example is $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$. We can apply the same logic to the verb-to-agent-noun relationship of ‘drive’: ‘driver’ and ‘swim’: ‘swimmer’.

Vector Equation

The relationship can be expressed with the following vector equation:

$$\vec{driver} - \vec{drive} \approx \vec{swimmer} - \vec{swim}$$

To find the vector for ”swimmer”, we rearrange the equation:

$$\vec{swimmer} \approx \vec{swim} - \vec{drive} + \vec{driver}$$

Given Vectors

- ‘drive’: [0.3, 0.4]
- ‘driver’: [0.8, 0.7]
- ‘swim’: [0.4, 0.1]

Calculation

We substitute the given vectors into the rearranged equation:

$$\begin{aligned}\vec{swimmer} &\approx [0.4, 0.1] - [0.3, 0.4] + [0.8, 0.7] \\ &= [0.4 - 0.3 + 0.8, \quad 0.1 - 0.4 + 0.7] \\ &= [0.1 + 0.8, \quad -0.3 + 0.7] \\ &= [0.9, \quad 0.4]\end{aligned}$$

The calculated word vector for **swimmer** is **[0.9, 0.4]**.